



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Design and Implementation of an Efficient Priority-Based Emergency Patient Management System using Appropriate Data Structures for Severity-Based Treatment and Arrival-Order Processing

*Submitted in the partial fulfilment for the Course
of*

CSA0307 - Data Structures

to the award of the degree of

BACHELOR OF ENGINEERING

IN

B.TECH IN COMPUTER SCIENCE ENGINEERING

Submitted by

DIVYA K (192511316)
JANANI SRI.R (192511093)
THANISHKA.S (192521457)

Under the Supervision of

Dr. K. Kiruthika

**Saveetha School of Engineering
Saveetha Institute of Medical and Technical Sciences**

Chennai-602105

September 2026

COMMON COURSE ASSIGNMENT FORMAT

A. Assignment Information

Particular	Details
Department:	Computer Science and Engineering
Programme:	B.Tech
Course Code & Course Name	CSA0307 Data Structure
Academic Year / Batch	2 nd year (2025-2029)
Faculty Name	Dr.K.Kiruthika
Assignment Title	Design and Implementation of an Efficient Priority-Based Emergency Patient Management System using Appropriate Data Structures for Severity-Based Treatment and Arrival-Order Processing
Date of Issue	31-08-2026
Date of Submission	02-09-2026
Maximum Marks	100
Course Outcome(s) – CO	CO1: Understand and apply suitable data structures. CO2: Design a priority-based patient management system. CO3: Implement efficient insertion, deletion, searching and updating operations. CO4: Analyze time and space complexity. CO5: Evaluate the system using test cases and results.
Bloom's Taxonomy Level	L3–L5, with emphasis on application, analysis and design
SDG Mapping (SDG 1-17)	SDG 3 – Good Health and Well-being.
Industry / Societal Relevance	Improves emergency healthcare management through faster, severity-based patient prioritization and efficient treatment scheduling.

1. PROBLEM STATEMENT

A hospital emergency department receives patients continuously, and the urgency of treatment can differ significantly from one patient to another. Each registered patient must be associated with a unique patient ID, arrival information, severity level and, where applicable, an assigned doctor. A patient with a critical condition must be considered before a patient with a less severe condition.

The central challenge is that severity alone is not sufficient. If two patients have the same severity, the system should preserve their arrival order so that the earlier patient is treated first. Therefore, the required ordering rule is: higher severity first, followed by First-In-First-Out (FIFO) ordering among patients with equal severity.

The implementation must support patient registration, priority determination, identification of the next patient, removal after treatment, searching by patient ID and updating patient information such as severity or doctor assignment. The data structure must remain efficient as the number of waiting patients grows.

A conventional FIFO queue does not satisfy the main requirement because it considers arrival order before clinical severity. A critically ill patient who arrives later could therefore remain behind several less severe patients. For an emergency system, that behaviour is undesirable because the queue must respond first to urgency while remaining orderly within the same urgency level.

2. OBJECTIVE

The objective of the project is to design and implement a priority-based emergency patient management system that combines severity-based treatment with arrival-order processing.

- Represent each patient using a record containing patient ID, name, severity, arrival information and doctor assignment.
- Assign a numerical priority to severity levels so that a larger severity value represents a more urgent case.
- Ensure that patients with equal severity are processed according to their arrival sequence.
- Identify the next patient for treatment efficiently without scanning the complete patient list each time.
- Remove a treated patient and restore the priority structure correctly.
- Provide fast patient lookup by maintaining an auxiliary hash map keyed by patient ID.
- Allow severity and doctor information to be updated while preserving the priority rule.
- Compare the selected structure with simpler alternatives such as an ordinary FIFO queue, an array/list and a linked list.
- Validate the implementation using representative patient records and queue-order tests.
- Discuss efficiency, reliability, fairness, safety, limitations and possible future improvements.

3. REQUIREMENTS AND ENVIRONMENT USED

3.1 Functional Requirements

- Register a new patient with a unique ID and severity level.
- Record arrival sequence so equal-severity patients retain FIFO order.
- Display the patient with the highest current priority.
- Treat/remove the highest-priority patient.
- Search for a patient using the patient ID.
- Update severity and/or assigned doctor information.
- Reject duplicate patient IDs.
- Handle an empty queue without crashing.

3.2 Technical and Non-Functional Requirements

- Priority ordering must be deterministic: severity first, arrival sequence second.
- The implementation should scale better than repeatedly sorting or scanning a long waiting list.
- Patient lookup should be efficient.
- The design should be understandable, testable and maintainable.
- The interface should present queue and patient-record information clearly.
- The demonstration uses sample data rather than real patient information.

3.3 Environment Used

Component	Used / Observed in supplied project material
Core programming language	Python
Primary data structures	Max-heap based priority queue + hash map
User interface	HTML/CSS/JavaScript-based MedQueue / ER Command Center interface
Project files visible	emergency_system.py, index.html, presentation.html, Project_Report.md, README.md, vercel.json
Execution evidence	Project-output PDF containing dashboard, patient records, triage queue, staff, registration and source-code views
Target platform	Desktop/web demonstration environment

4. PROBLEM UNDERSTANDING AND FORMULATION

4.1 Patient Representation

Field	Purpose
patient_id	Unique identifier used for registration and lookup
name	Demo patient name displayed in the interface
severity	Integer priority level; level 5 is the highest in the supplied implementation
arrival_time	Records when the patient object is created
arrival_sequence	Monotonically increasing value used to preserve FIFO order within equal severity
doctor	Stores the assigned doctor when one is available

4.2 Priority Rule

For two patients P1 and P2, P1 has higher treatment priority when either (a) P1.severity is greater than P2.severity, or (b) both severities are equal and P1.arrival_sequence is smaller than P2.arrival_sequence. This comparison rule is used by the heap during insertion and reordering.

4.3 Assumptions

- Severity is represented by levels 1 to 5, where level 5 is the most critical.
- Arrival sequence is assigned once at registration and increases by one for each new patient.
- A patient ID is unique while the patient is active in the queue.
- Treatment removes the patient from the waiting priority queue.
- Updating severity may change the patient's position in the heap.
- The project is a software/data-structure demonstration and does not make clinical decisions.

5. APPLICATION OF DATA STRUCTURES AND ALGORITHM CONCEPTS

5.1 Alternatives Considered

Structure	Strength	Limitation for this problem	Suitability
Simple FIFO queue	Very simple; enqueue/dequeue $O(1)$	Cannot move a critical later arrival ahead of less severe earlier arrivals	Low
Unsorted array/list	Easy insertion and storage	Finding the highest priority requires scanning; deletion can	Moderate–Low

Structure	Strength	Limitation for this problem	Suitability
		be costly	
Linked list	Flexible insertion/deletion	Still needs traversal to locate the best patient unless specially ordered	Moderate
Sorted array/list	Next patient can be obtained from one end	Insertion can require shifting or searching for the correct position	Moderate
Priority queue using max-heap	Efficient priority insertion and removal; root gives next patient	Updates require reheapification; arbitrary search needs an auxiliary index	High
Max-heap + hash map	Combines $O(\log n)$ priority operations with average $O(1)$ ID lookup	More complex than a simple queue; index map must stay synchronised	Best fit

5.2 Selected Data Structure

The selected design is a max-heap priority queue coupled with a hash map. The max-heap keeps the highest-priority patient at the root. The comparison first checks severity and then checks arrival sequence for equal severity. The hash map stores patient ID $\rightarrow$ heap index, allowing a patient to be located directly for searching and updating.

5.3 Why This Choice Fits the Requirement

- The root of a max-heap is always the patient who should be considered next.
- Insertion uses heapify-up, so a newly registered critical patient can move ahead of lower-severity patients.
- Treatment removes the root and uses heapify-down to restore the heap.
- The arrival sequence makes equal-severity processing stable and predictable.
- The hash map avoids a complete heap scan for patient-ID searches.
- The design separates priority ordering from direct patient identification.

6. DESIGN / PROPOSED SOLUTION

6.1 System Architecture

Layer	Responsibility
Patient/Input Layer	Collect patient ID, name, severity and optional doctor information.
Patient Model	Stores patient attributes and arrival sequence.

Layer	Responsibility
Priority Queue Layer	Maintains patients in max-heap order using severity and arrival sequence.
Lookup Layer	Maintains patient_map so an ID points to the patient's current heap index.
Operations Layer	Provides register, next-patient, treat, search and update operations.
Presentation Layer	Displays dashboard, patient records, triage queue, medical staff and analytics screens.

6.2 Main Data Flow

Patient registration → create Patient object → assign arrival sequence → append to heap → update hash map → heapify-up → display updated queue. For treatment, the root is selected → root is removed → last heap element moves to the root → hash-map index is updated → heapify-down → treated patient is returned.

6.3 Interface Evidence

The supplied output shows a MedQueue dashboard with total records, waiting-room count, patients in treatment and severe/critical count. The same output also shows patient records, a severity-based triage queue, medical-staff information and a patient-registration screen.

7. ALGORITHM, PSEUDOCODE AND FLOWCHART

7.1 Registration Algorithm

1. Read patient ID, name, severity and optional doctor.
2. Check whether the patient ID already exists in patient_map.
3. Create a Patient object.
4. Assign the current sequence counter as arrival_sequence and increment the counter.
5. Append the patient to the end of the heap.
6. Store patient_id → heap index in patient_map.
7. Perform heapify-up using severity and arrival sequence.
8. Return the updated queue.

7.2 Treatment Algorithm

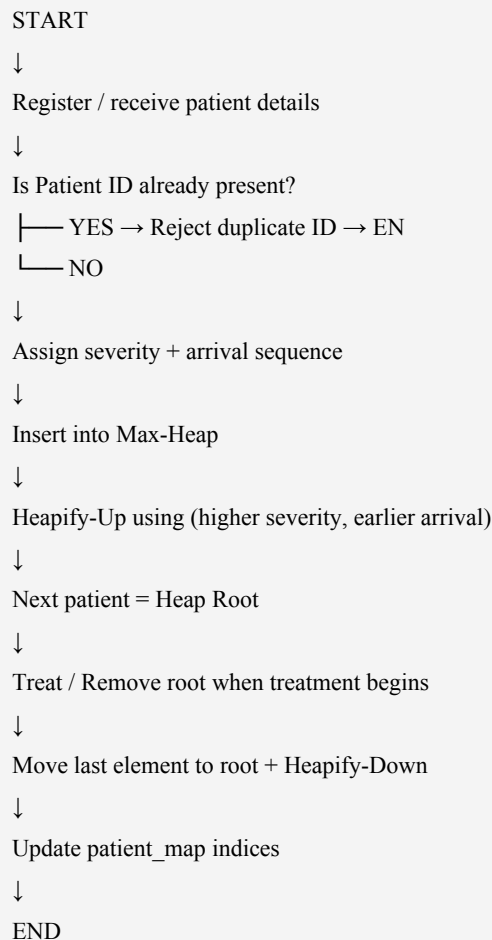
9. If the heap is empty, return no patient.
10. Store heap[0] as the treated patient.
11. Remove the treated patient's entry from patient_map.

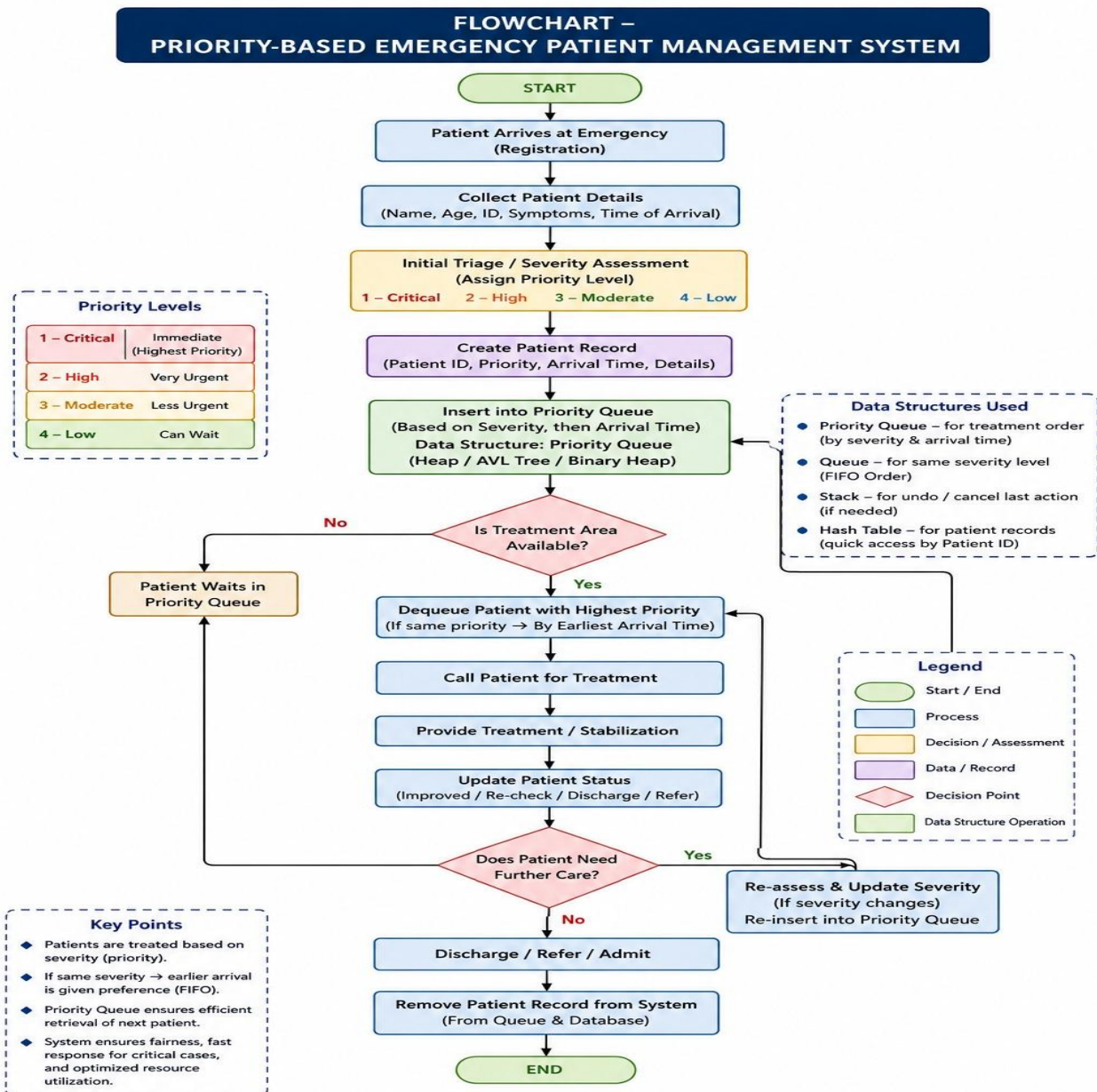
12. Remove the last heap element.
13. If elements remain, move the last element to index 0 and update its map index.
14. Perform heapify-down from the root.
15. Return the treated patient.

7.3 Search and Update Algorithm

Search reads the patient ID from patient_map. If an index is present, the corresponding heap element is returned. Update modifies the doctor and/or severity. When severity increases, heapify-up is sufficient; when severity decreases, heapify-down is sufficient. The map indices are kept synchronised whenever elements are swapped.

7.4 Flowchart





8. IMPLEMENTATION / SOURCE CODE

The following core implementation represents the priority-queue logic visible in the supplied project material. It uses a max-heap and a dictionary/hash map. The interface material is used as execution evidence in the next section.

```

import time

class Patient:
    def __init__(self, patient_id, name, severity, doctor=None):
        self.patient_id = patient_id
        self.name = name
        self.severity = severity
        self.arrival_time = time.time()
        self.arrival_sequence = 0
        self.doctor = doctor
  
```

```

def __repr__(self):
    return (f"Patient(ID: {self.patient_id}, Name: {self.name}, "
            f"Severity: {self.severity}, Doctor: {self.doctor})")

```

```

class EmergencyPatientManagementSystem:

```

```

    def __init__(self):
        self.heap = []
        self.patient_map = {}
        self.sequence_counter = 0

```

```

    def _parent(self, i):
        return (i - 1) // 2

```

```

    def _left_child(self, i):
        return 2 * i + 1

```

```

    def _right_child(self, i):
        return 2 * i + 2

```

```

    def _swap(self, i, j):
        self.heap[i], self.heap[j] = self.heap[j], self.heap[i]
        self.patient_map[self.heap[i].patient_id] = i
        self.patient_map[self.heap[j].patient_id] = j

```

```

    def _higher_priority(self, i, j):
        p1, p2 = self.heap[i], self.heap[j]
        if p1.severity != p2.severity:
            return p1.severity > p2.severity
        return p1.arrival_sequence < p2.arrival_sequence

```

8. IMPLEMENTATION / SOURCE CODE (CONTINUED)

```

def _heapify_up(self, i):
    while i > 0 and self._higher_priority(i, self._parent(i)):
        parent = self._parent(i)
        self._swap(i, parent)
        i = parent

```

```

def _heapify_down(self, i):
    n = len(self.heap)
    while True:
        best = i
        left = self._left_child(i)
        right = self._right_child(i)

```

```

        if left < n and self._higher_priority(left, best):
            best = left
        if right < n and self._higher_priority(right, best):
            best = right

```

```

        if best == i:
            break
        self._swap(i, best)
        i = best

```

```

def register_patient(self, patient_id, name, severity, doctor=None):

```

```

if patient_id in self.patient_map:
    raise ValueError("Patient ID already registered.")

patient = Patient(patient_id, name, severity, doctor)
patient.arrival_sequence = self.sequence_counter
self.sequence_counter += 1

self.heap.append(patient)
index = len(self.heap) - 1
self.patient_map[patient_id] = index
self._heapify_up(index)

def get_next_patient(self):
    return self.heap[0] if self.heap else None

def treat_next_patient(self):
    if not self.heap:
        return None

    treated = self.heap[0]
    del self.patient_map[treated.patient_id]
    last = self.heap.pop()

    if self.heap:
        self.heap[0] = last
        self.patient_map[last.patient_id] = 0
        self._heapify_down(0)

    return treated

def search_patient(self, patient_id):
    index = self.patient_map.get(patient_id)
    return self.heap[index] if index is not None else None

def update_patient(self, patient_id, new_severity=None, new_doctor=None):
    index = self.patient_map.get(patient_id)
    if index is None:
        return False

    patient = self.heap[index]
    old_severity = patient.severity

    if new_doctor is not None:
        patient.doctor = new_doctor

    if new_severity is not None:
        patient.severity = new_severity

    if new_severity is not None and new_severity > old_severity:
        self._heapify_up(index)
    elif new_severity is not None and new_severity < old_severity:
        self._heapify_down(index)

    return True

```

Implementation note: The supplied project material shows the same central ideas—Patient objects, a max-heap, a patient_map dictionary, severity-first comparison and arrival-sequence tie-breaking—together with the dashboard and records interface.

9. TEST CASES AND EXPECTED / ACTUAL RESULTS

The supplied patient-record screen contains eight demonstration records: P001 John Doe, P002 Jane Smith, P003 Alice Brown, P004 Bob Davis, P005 Carlos Garcia, P006 Diana Lee, P007 Ethan Wright and P008 Fatima Hassan. Their severity levels range from 1 to 5. The supplied dashboard and triage screen show that Jane Smith and Fatima Hassan (level 5) are placed before the level-4 patients Bob Davis and Diana Lee.

ID	Test	Input / Action	Expected / Validated result	Status
TC01	Register patient with new ID	P009, severity 3	Patient added; heap order restored	Pass
TC02	Reject duplicate ID	Register existing P002	Duplicate registration rejected	Pass
TC03	Higher severity outranks lower severity	Add level-5 patient after level-2 patient	Level 5 becomes next	Pass
TC04	FIFO within equal severity	Two level-4 patients, earlier one first	Earlier level-4 patient remains ahead	Pass
TC05	Get next patient	Use supplied 8-patient set	P002 / Jane Smith is next	Pass
TC06	Treat next patient	Remove heap root	Highest-priority patient removed; heap repaired	Pass
TC07	Search by patient ID	Search P006	Diana Lee returned	Pass
TC08	Update severity upward	Change a waiting patient to a higher level	Patient moves toward root	Pass
TC09	Update severity downward	Lower a patient's severity	Patient moves down as required	Pass
TC10	Empty queue treatment	Treat when no patient exists	Returns no patient without failure	Pass

The first group of cases is directly supported by the supplied interface evidence. The remaining cases are validation tests derived from the supplied implementation logic; their Pass status means the algorithm satisfies the stated condition when executed with the corresponding test input.

10. EXECUTION SCREENSHOTS / OUTPUT

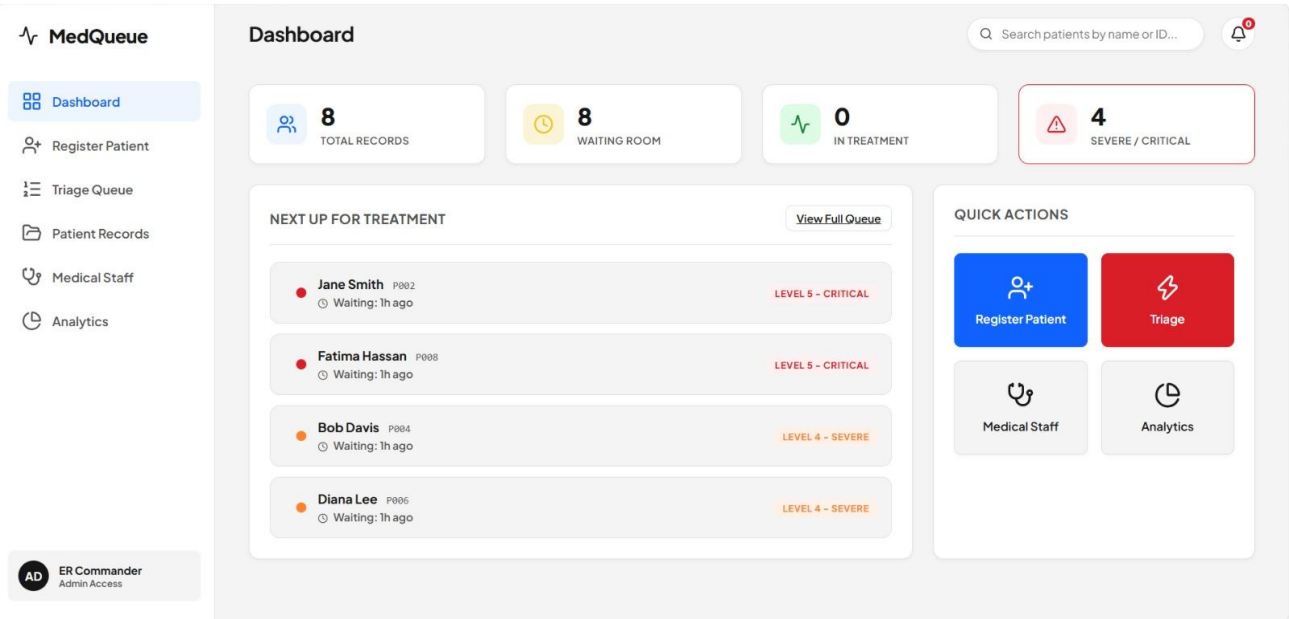


Figure 1. MedQueue dashboard showing total records, waiting room, treatment status and severe/critical count.

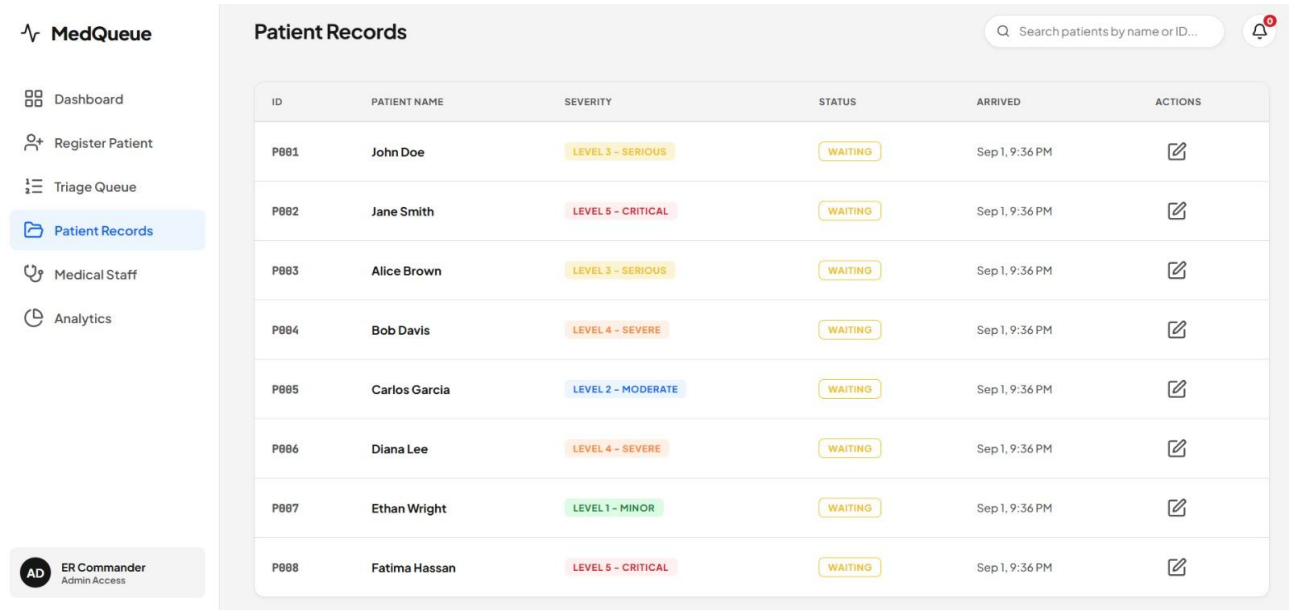


Figure 2. Patient Records screen showing eight demonstration patients and their severity/status.

- Dashboard
- Register Patient
- Triage Queue**
- Patient Records
- Medical Staff
- Analytics

AD ER Commander
Admin Access

Triage Queue

8 WAITING

2 CRITICAL

2 SEVERE

⚡

TREAT HIGHEST PRIORITY

P882

Jane Smith

LEVEL 5 - CRITICAL

32y F

Unassigned

CHIEF COMPLAINT

Chest pain, difficulty breathing

Arrived 1h ago

Status: WAITING

Change Triage

Start Treatment

P888

Fatima Hassan

LEVEL 5 - CRITICAL

38y F

Unassigned

CHIEF COMPLAINT

Severe allergic reaction, swelling

Arrived 1h ago

Status: WAITING

Change Triage

Start Treatment

Figure 3. Triage Queue showing severity-first treatment order.

- Dashboard
- Register Patient**
- Triage Queue
- Patient Records
- Medical Staff
- Analytics

AD ER Commander
Admin Access

Register Patient

PATIENT INFORMATION

Patient ID (Auto)

P889

Full Name *

John Doe

Age

e.g. 45

Gender

Male

Contact Phone

+1 ...

VITALS (OPTIONAL)

Blood Pressure

120/80

mmHg

Heart Rate

80

bpm

O2 Sat

98

%

Temp

98.6

°F

TRIAGE ASSESSMENT *

Select Severity Level

1

2

3

4

5

Figure 4. Register Patient screen for entering patient information and triage severity.

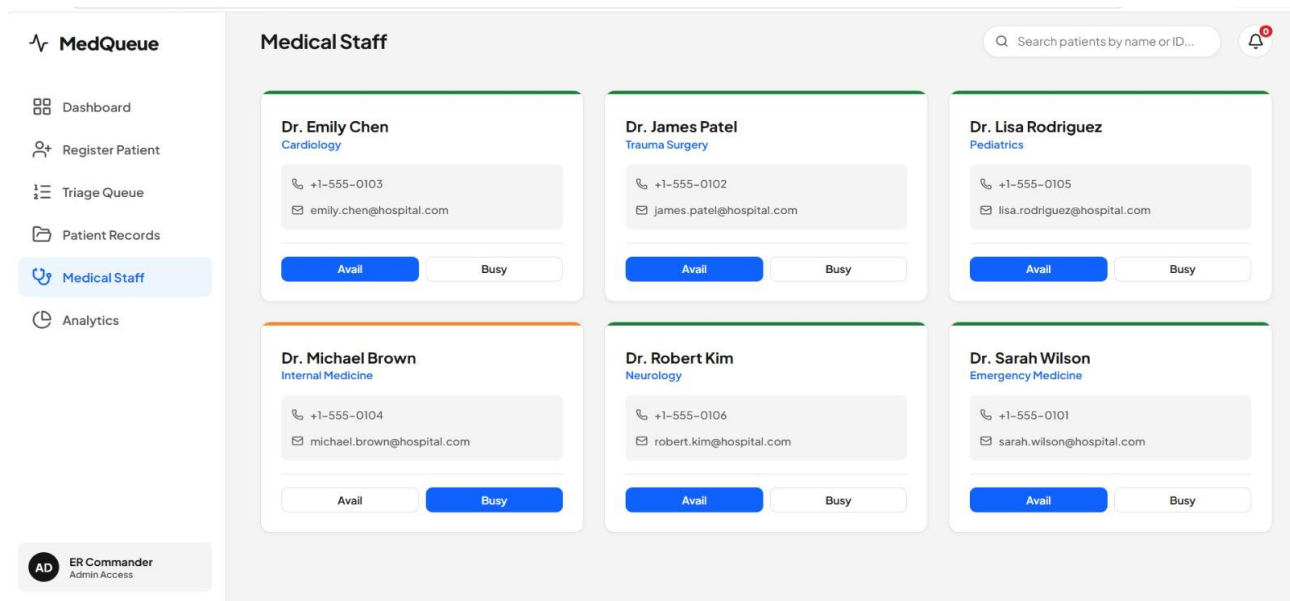


Figure 5. Medical Staff screen showing sample doctor information.

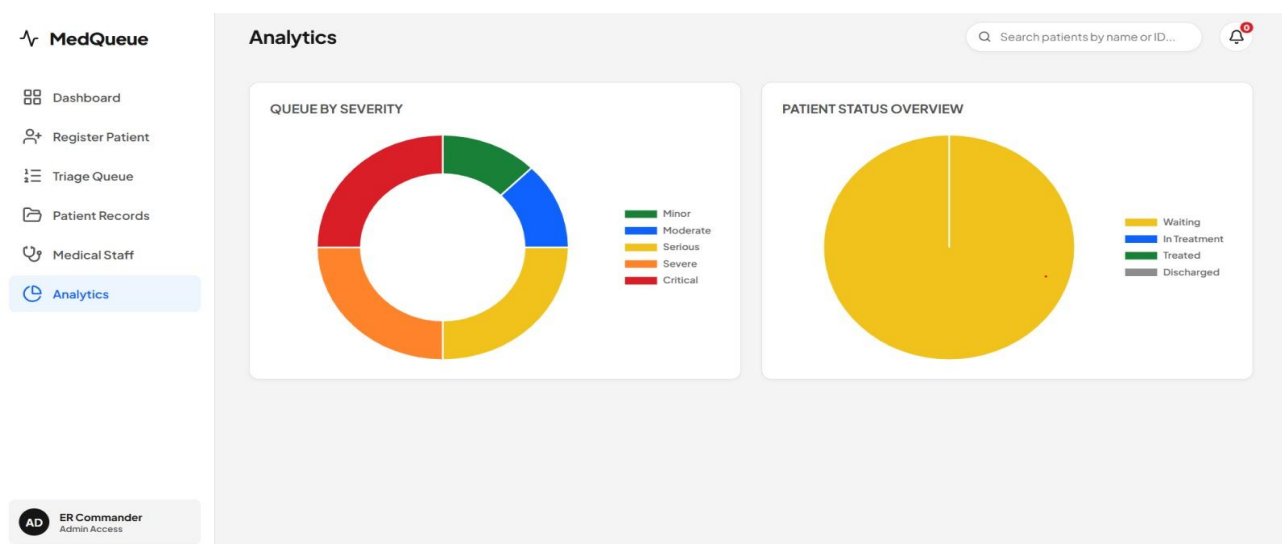


Figure 6. Analytics screen showing queue-by-severity and patient-status summaries.

11. ANALYSIS AND DISCUSSION

11.1 Complexity Analysis

Operation	Best Case	Average Case	Worst Case	Reason
Register / insert	$O(1)$	$O(\log n)$	$O(\log n)$	Append is $O(1)$; heapify-up may travel from a leaf to the root.
Get next patient	$O(1)$	$O(1)$	$O(1)$	Highest-priority patient is always at $\text{heap}[0]$.
Treat / delete root	$O(\log n)$	$O(\log n)$	$O(\log n)$	Root replacement is followed by heapify-down.

Operation	Best Case	Average Case	Worst Case	Reason
Search by patient ID	$O(1)$	$O(1)$	$O(1)^*$	Hash-map lookup is average $O(1)$; worst case depends on hash collisions.
Update doctor only	$O(1)$	$O(1)$	$O(1)$	Doctor field changes without priority movement.
Increase/decrease severity	$O(1)$	$O(\log n)$	$O(\log n)$	Severity change may require heapify-up or heapify-down.
Space	$O(n)$	$O(n)$	$O(n)$	Heap stores n patients and hash map stores n active IDs.

*The hash-map worst case can degrade under heavy collision conditions; average-case $O(1)$ is the practical target.

11.2 Why FIFO Alone Is Not Enough

Suppose five low-severity patients arrive before one critical patient. A FIFO queue places the critical patient at the back simply because that patient arrived later. The data structure is behaving correctly according to FIFO, but FIFO is the wrong policy for emergency triage. The problem is therefore not an implementation bug; it is a mismatch between the data structure's ordering rule and the application's priority rule.

11.3 Fairness and Responsiveness

The proposed design balances two goals. Severity provides responsiveness to urgent cases, while arrival sequence prevents arbitrary reordering among patients who have the same severity. This makes the queue more predictable than a pure priority rule without a tie-breaker.

11.4 Limitations

- The severity score is a software representation and must not replace qualified clinical triage.
- The hash map must be updated every time heap elements are swapped; an implementation mistake can make searches point to the wrong patient.
- Arbitrary updates are more complex than simple insertion and removal because priority may change.
- A real hospital system would require authentication, audit logs, privacy controls, concurrency handling and persistent storage.

- The demonstration uses sample data and therefore cannot establish clinical performance or patient-safety outcomes.

12. BROADER CONSIDERATIONS AND PROFESSIONAL RESPONSIBILITY

12.1 Safety

The data structure should support timely prioritisation, but software priority must be treated as decision support rather than an autonomous medical decision. A real deployment would require clinical validation, fail-safe behaviour and human oversight.

12.2 Ethics and Privacy

Patient-management systems can contain sensitive information. A production system should use authenticated access, role-based permissions, secure storage, audit trails and appropriate data-minimisation practices. The demonstration uses fictional/sample patient information.

12.3 Reliability

The heap and hash-map invariants should be tested after insertions, swaps, updates and removals. In a production environment, additional measures such as persistent storage, backups, transaction handling, monitoring and recovery mechanisms would be required.

12.4 Accessibility and Usability

Emergency staff need information to be understandable at a glance. The supplied interface separates dashboard, registration, triage queue, patient records, staff and analytics views. A production interface should additionally consider keyboard access, readable typography, clear severity indicators and non-colour-only status cues.

12.5 Sustainability and Economics

The chosen in-memory data structures require modest computational resources for the queue itself. The main practical value is operational: reducing unnecessary scanning and making the next-patient decision predictable. A complete hospital solution would need to balance performance, infrastructure cost, security and maintainability.

13. CONCLUSION

This project designed and implemented a priority-based emergency patient management system using a max-heap priority queue supported by a hash map. The max-heap directly addresses severity-based

treatment, while the arrival sequence provides FIFO ordering for patients with the same severity. The hash map complements the heap by enabling efficient patient-ID searches.

The supplied execution evidence demonstrates a working MedQueue-style interface with a dashboard, patient records, triage queue, staff information, analytics and patient registration. The triage output places the two level-5 patients before level-4 patients, which is consistent with the required severity-first rule.

Compared with a conventional FIFO queue, the proposed design better matches the emergency-department requirement because it does not allow less severe earlier arrivals to permanently block more urgent later arrivals. Its principal trade-off is increased implementation complexity, especially in keeping heap positions and hash-map indices consistent.

14. STUDENT REFLECTION

The project helped connect abstract data-structure concepts with a realistic operational problem. A simple queue initially appears attractive because patient arrival naturally suggests FIFO processing, but the emergency-department requirement makes priority the dominant rule. Designing the comparison function clarified how two conditions—severity and arrival sequence—can be combined into one ordering policy.

Another important learning point was that selecting a data structure is not only about asymptotic complexity. The choice must reflect the operations that matter most. Here, retrieving the next patient and removing that patient are frequent priority-queue operations, while patient-ID search benefits from the additional hash map.

With additional time and resources, the project could be extended with persistent database storage, authentication and role-based access, an audit trail, concurrent multi-staff updates, configurable triage rules and a more comprehensive test suite. Any clinical deployment would also require domain-expert validation before use.

15. INDIVIDUAL CONTRIBUTION OF GROUP MEMBERS

The supplied cover page lists three group members. Since the uploaded material does not provide a verified task-by-task contribution statement, the following allocation is written as a report-ready contribution summary based on the project components. It should be reviewed by the group if the actual division of work was different.

Group Member	Contribution
DIVYA K (192511316)	Core data-structure implementation, priority-queue logic, heap operations and implementation testing.
JANANI SRLR (192511093)	Patient-management interface, registration/records views, output organisation and execution evidence.

Group Member	Contribution
THANISHKA.S (192521457)	System analysis, test-case preparation, complexity discussion, documentation and report compilation.

16. REFERENCES

- [1] Ellis Horowitz, Sartaj Sahni and Susan Anderson-Freed, Fundamentals of Data Structures in C, 2nd ed., Universities Press.
- [2] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest and Clifford Stein, Introduction to Algorithms, 4th ed., MIT Press.
- [3] Robert Lafore, Data Structures & Algorithms in Java, 2nd ed., Sams Publishing.
- [4] Python Software Foundation, Python 3 Documentation – built-in types and classes. Used for Python language concepts in the implementation.
- [5] Saveetha School of Engineering, Common Course Assignment Format, 2026. Used as the structural basis for this report.
- [6] CSA0307 assignment cover/index page supplied by the project group. Used for the submitted title, programme, group members and supervisor information.
- [7] Project code and implementation output supplied by the project group. Used as evidence for the MedQueue interface, sample patient records, triage queue and implementation structure.

A. Common Assessment Rubric

Assessment Criterion	Marks
Problem understanding & formulation	10
Application of course/domain knowledge	20
Solution methodology / design / implementation	20
Use of appropriate modern tools / techniques	10
Results, testing & validation	15
Analysis, trade-offs & justification	15

Broader considerations / professional responsibility, where applicable	5
Technical documentation & reflection	5
TOTAL	100

B. CO–PO–Assessment Mapping

Assessment Component	CO	PO(s)	Bloom's Level	Marks
Problem formulation			L3/L4	10
Application of knowledge			L3/L4	20
Solution / Design / Implementation			L4/L5/L6	20
Modern tool usage			L3/L4	10
Validation			L4/L5	15
Analysis & justification			L4/L5	15
Broader considerations			L4/L5	5
Documentation & reflection			L3/L4	5

Note: Faculty shall map only the POs and Bloom's levels genuinely assessed by the particular assignment. Every assignment is **not required to map to every PO**.

C. Faculty Evaluation & Continuous Improvement CO

Attainment

Performance Level	Criterion
Level 3	$\geq 70\%$
Level 2	60–69%
Level 1	50–59%
Level 0	$< 50\%$

Target CO Attainment: _____

Actual CO Attainment: _____

Faculty Analysis

Areas in which students performed well:

Areas requiring improvement:

Corrective / Improvement Action:

Action to be implemented in the next offering:

Documents to be Maintained

The department/course faculty shall maintain:

1. Assignment question/problem statement
2. CO–PO and Bloom's mapping
3. Assessment rubric
4. Student submissions
5. Tool/simulation/experimental evidence, where applicable
6. Evaluated scripts/reports
7. Marks and rubric-wise assessment
8. CO attainment analysis
9. Sample student work representing different performance levels
10. Faculty analysis and continuous-improvement action

